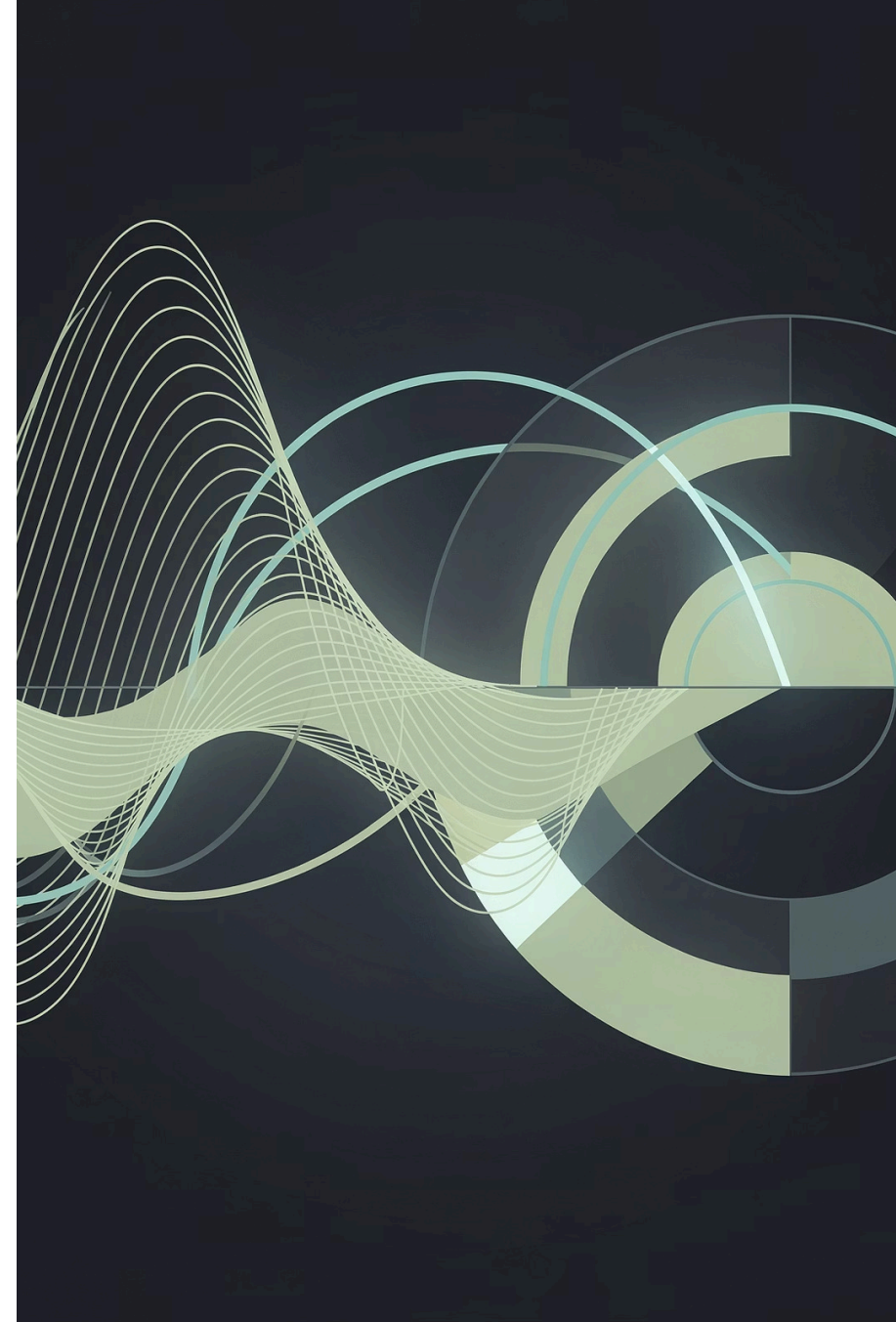


UNLOX

# The Fundamentals of Regression

From linear predictions to regularization — a deep dive into the algorithms that power modern machine learning.

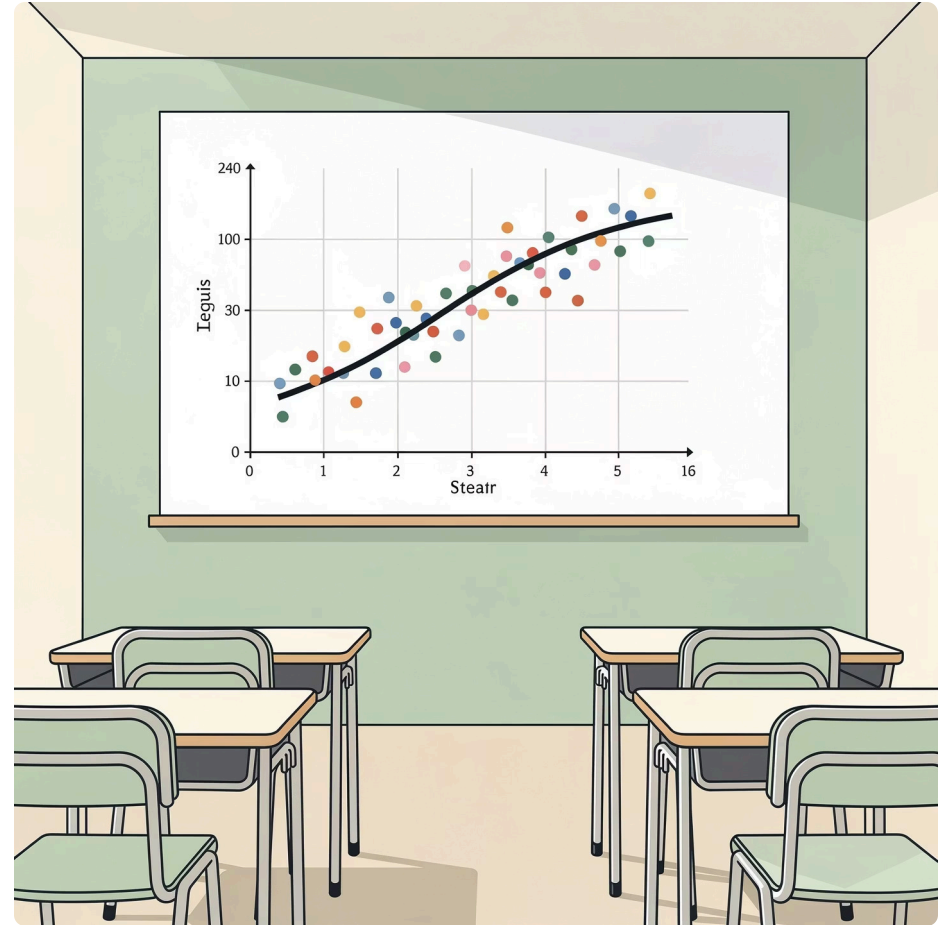


# What Is Regression?

At its core, **regression** is the task of predicting a continuous or categorical outcome from input data. It forms the backbone of supervised learning and is often the first family of algorithms that practitioners encounter.

Regression models learn a mathematical relationship between **features** (inputs) and a **target variable** (output). Once trained, they can generalize this relationship to make predictions on unseen data — the hallmark of useful machine learning.

In this module, we'll explore two foundational regression algorithms, the optimization process that powers them, and the regularization techniques that keep them honest.



# Linear Regression

**Linear Regression** is the foundational algorithm for predicting a **continuous numerical value** — think house prices, stock returns, or temperature forecasts. It works by fitting the best straight line (or hyperplane, in higher dimensions) through your data, capturing the relationship between input features and the output variable.

1

## Collect Features

Gather input variables (e.g., square footage, number of bedrooms) that may influence the target.

2

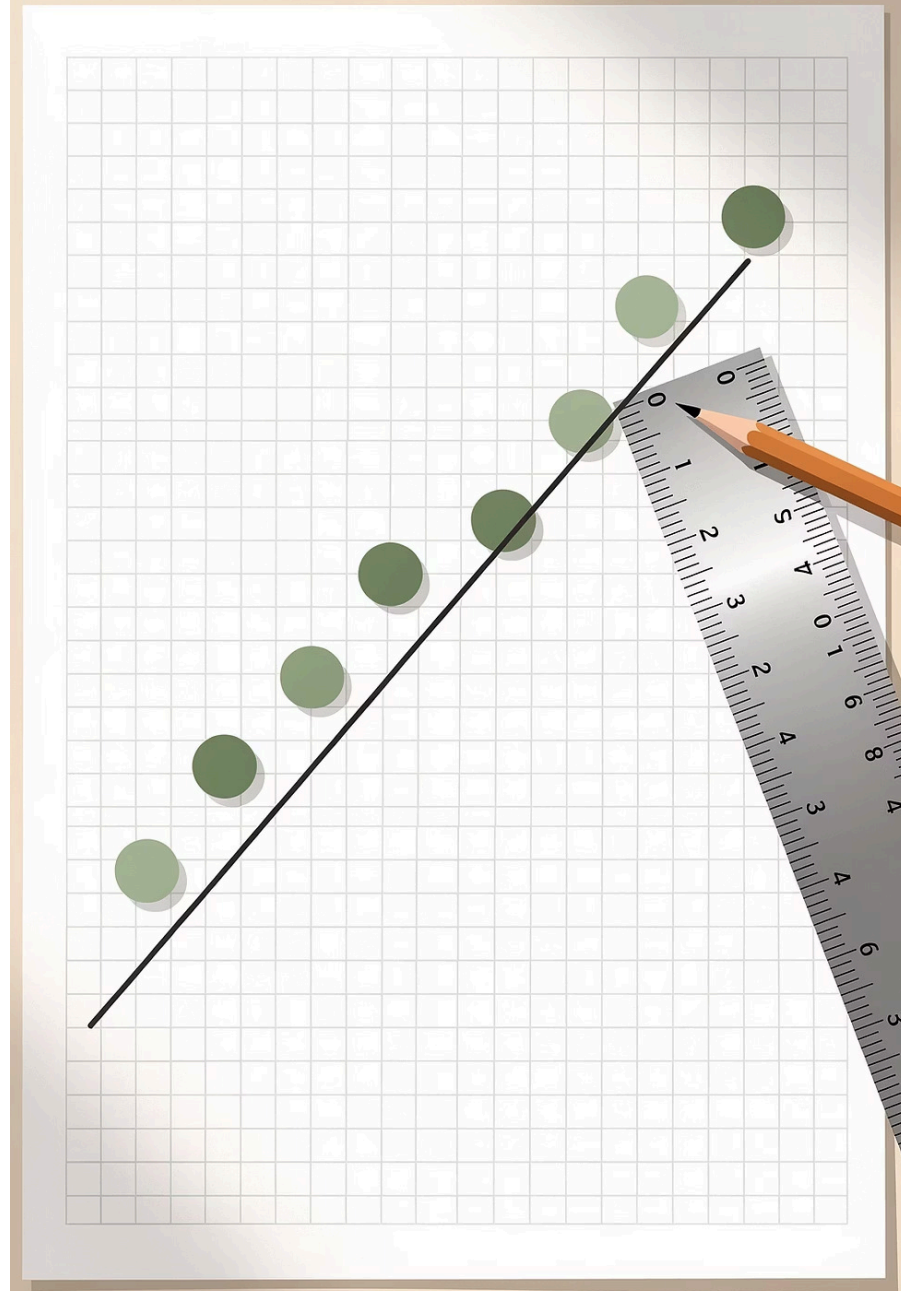
## Fit the Line

The algorithm finds the slope (weight) and intercept (bias) that minimize prediction error across all training examples.

3

## Make Predictions

New inputs are plugged into the learned equation  $\hat{y} = \mathbf{w}x + \mathbf{b}$  to produce continuous predictions.



# Logistic Regression

Despite its name, **Logistic Regression** is a **classification** algorithm — not a regression one. It is used to predict categorical outcomes, most commonly binary labels like "spam vs. not spam" or "approved vs. denied." The key insight is the **logistic (sigmoid) function**, which squashes a linear equation's output into a probability between 0 and 1.

## How It Works

A linear combination of features is computed, then passed through the sigmoid function:

$$\sigma(z) = 1 / (1 + e^{-z})$$

The resulting probability is thresholded (typically at 0.5) to assign a class label. Values above the threshold map to class 1; values below map to class 0.

## When to Use It

- Binary classification tasks (yes/no, true/false)
- When you need interpretable probability outputs
- As a baseline before trying more complex classifiers
- Multi-class problems via one-vs-rest or softmax extensions

# Linear vs. Logistic — Side by Side

Understanding when to reach for each algorithm is a critical skill. The table below highlights the essential differences between these two foundational methods.

| Characteristic   | Linear Regression        | Logistic Regression          |
|------------------|--------------------------|------------------------------|
| Task Type        | Regression (continuous)  | Classification (categorical) |
| Output           | Real-valued number       | Probability (0 to 1)         |
| Activation       | None (identity)          | Sigmoid function             |
| Loss Function    | Mean Squared Error (MSE) | Binary Cross-Entropy         |
| Example Use Case | Predicting house price   | Email spam detection         |

 Both algorithms share a linear core — the difference lies in how the output is transformed and interpreted.

UNLOX

CHAPTER 7.2

# Model Building & Optimization

Finding the optimal parameters for a regression model — the weights and biases that minimize prediction error — is the essence of **model building**. This process transforms a generic equation into a trained, predictive model tailored to your specific data.

The workhorse behind this optimization is an algorithm called **Gradient Descent**, which iteratively nudges parameters in the direction that reduces error the fastest.





# How Gradient Descent Works

Imagine standing on a hilly landscape in thick fog. You can't see the valley floor, but you can feel the slope beneath your feet. **Gradient Descent** works exactly like this — it computes the slope (gradient) of the error surface at the current position and takes a step downhill.



## Initialize

Start with random parameter values for weights and bias.



## Compute Gradient

Calculate the partial derivative of the loss with respect to each parameter.



## Update Parameters

Adjust each parameter by subtracting the gradient scaled by the **learning rate** ( $\alpha$ ).



## Converge

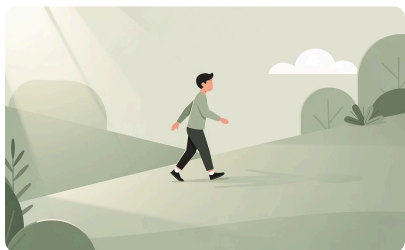
Repeat until the loss stabilizes at a minimum — the model is now optimized.

The **learning rate** is critical: too large and the algorithm overshoots the minimum; too small and convergence takes an impractically long time.

UNLOCK

LEARNING RATE IMPACT

# Choosing the Right Learning Rate



## Too Small ( $\alpha \ll 0.01$ )

The model converges extremely slowly, requiring thousands of extra iterations. Training becomes computationally expensive and may stall before reaching a good minimum.



## Just Right ( $\alpha \approx 0.01$ )

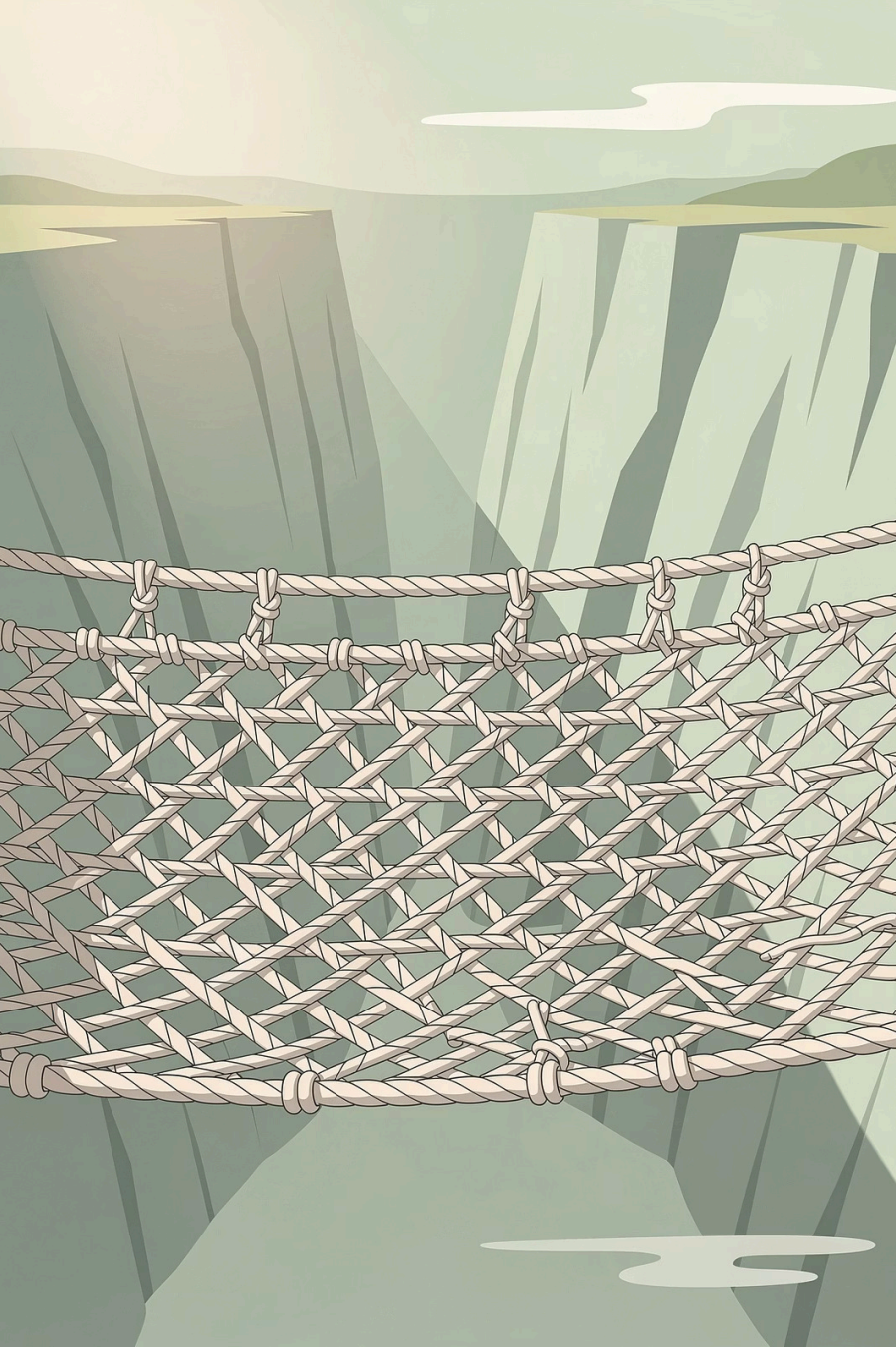
Parameters update at a pace that balances speed and stability. The model reaches a low-error minimum efficiently — this is the sweet spot practitioners aim for.



## Too Large ( $\alpha \gg 0.1$ )

Updates overshoot the minimum, causing the loss to oscillate wildly or even diverge. The model never settles on good parameters.





UNLOCK

CHAPTER 7.3

# Regularization — Keeping Models Honest

**Regularization** is a family of techniques designed to prevent **overfitting** — the phenomenon where a model memorizes training data noise instead of learning the underlying pattern. An overfit model performs brilliantly on training data but fails catastrophically on new, unseen examples.

Regularization works by adding a **penalty term** to the loss function that discourages excessively large coefficients. This nudges the model toward simpler, more generalizable solutions — trading a tiny amount of training accuracy for dramatically better real-world performance.

# L1 (Lasso) vs. L2 (Ridge) Regularization

The two most widely used regularization techniques each add a different type of penalty, leading to fundamentally different behaviors in how they shrink model coefficients.

## L1 — Lasso

Adds the **sum of absolute values** of coefficients to the loss. This drives some weights all the way to zero, effectively performing **feature selection**. Use L1 when you suspect many features are irrelevant.

## L2 — Ridge

Adds the **sum of squared values** of coefficients to the loss. This shrinks all weights toward zero but rarely eliminates any entirely. Use L2 when all features likely contribute, and you want to **reduce their collective magnitude**.

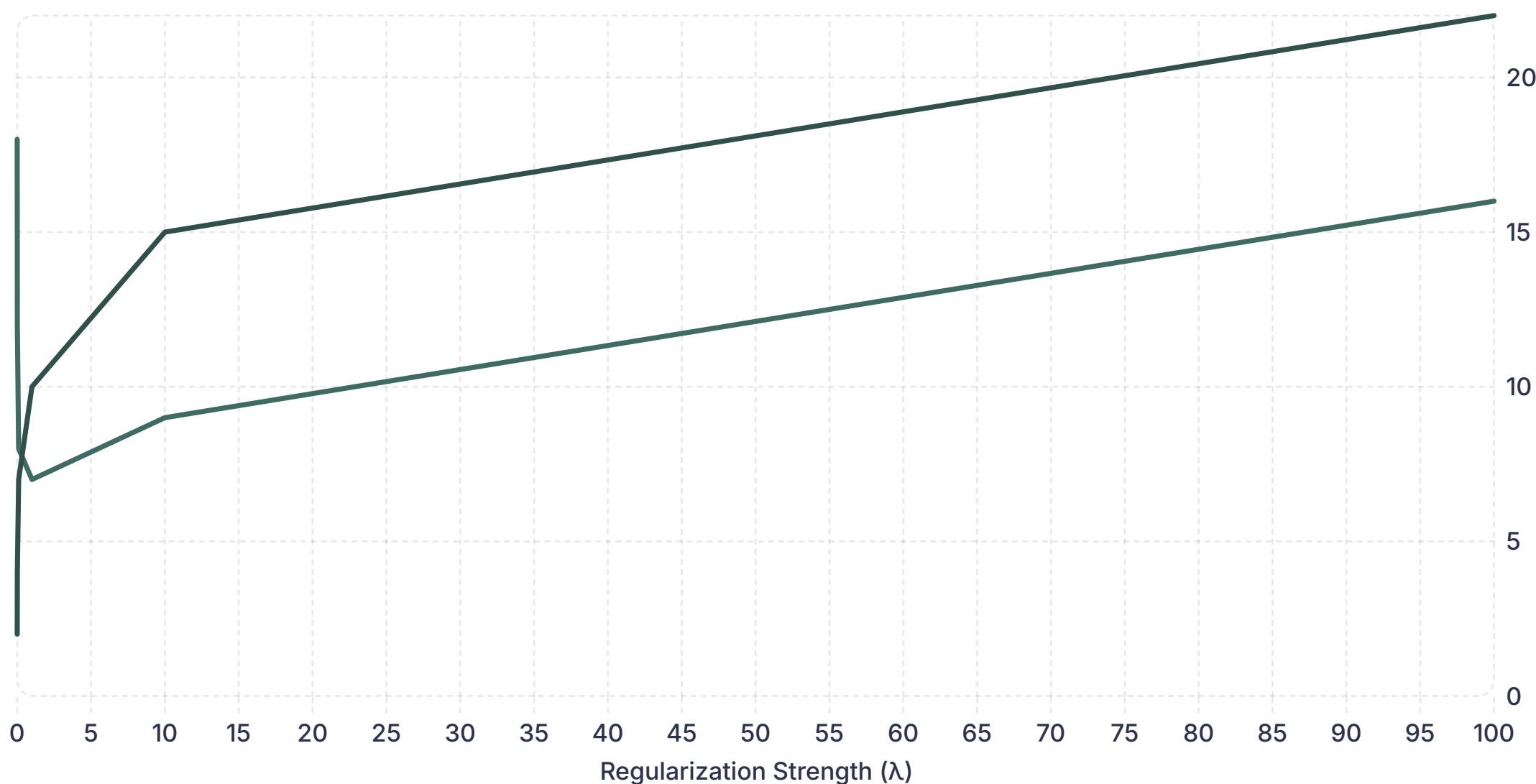
- ❏ **Elastic Net** combines both L1 and L2 penalties, offering a middle ground. It's especially useful when features are correlated — a scenario where pure Lasso can behave unpredictably.

# The Regularization Effect

Visualizing what happens to model complexity as regularization strength increases helps build intuition for why this technique is so essential.

— Test Error

— Training Error



At low regularization, the model overfits — training error is minimal but test error is high. As  $\lambda$  increases, the gap narrows until test error reaches its **sweet spot**. Push  $\lambda$  too far, and both errors climb — the model underfits. Finding the optimal  $\lambda$  is typically done via **cross-validation**.

# Putting It All Together



The regression pipeline follows a clear, logical sequence. First, understand whether your task requires predicting a continuous value or a class label. Then, select the appropriate model, optimize it with gradient descent, and apply regularization to ensure it generalizes well beyond the training set. Mastering this workflow gives you a solid foundation for tackling far more complex algorithms down the road.